MIPS 32 BIT MULTI-CYCLE PROCESSOR

To:

DR/ Howida Abdullatif

ENG/ Nourhan Abdelaziz

OUR TEAM

Essam Mohammed Essam Shenhab	20812022100664
Ali Mohamed Ali Abdallah	20812022100564
Omar Ahmed Abdelaziz Mashour	20812022100551
Merna Atef Ibrahim Ahmed	20812022201132

CONTENT

1 MIPS Architecture Overview

2 Datapath Design

3 Control Unit Design

4 Implementation

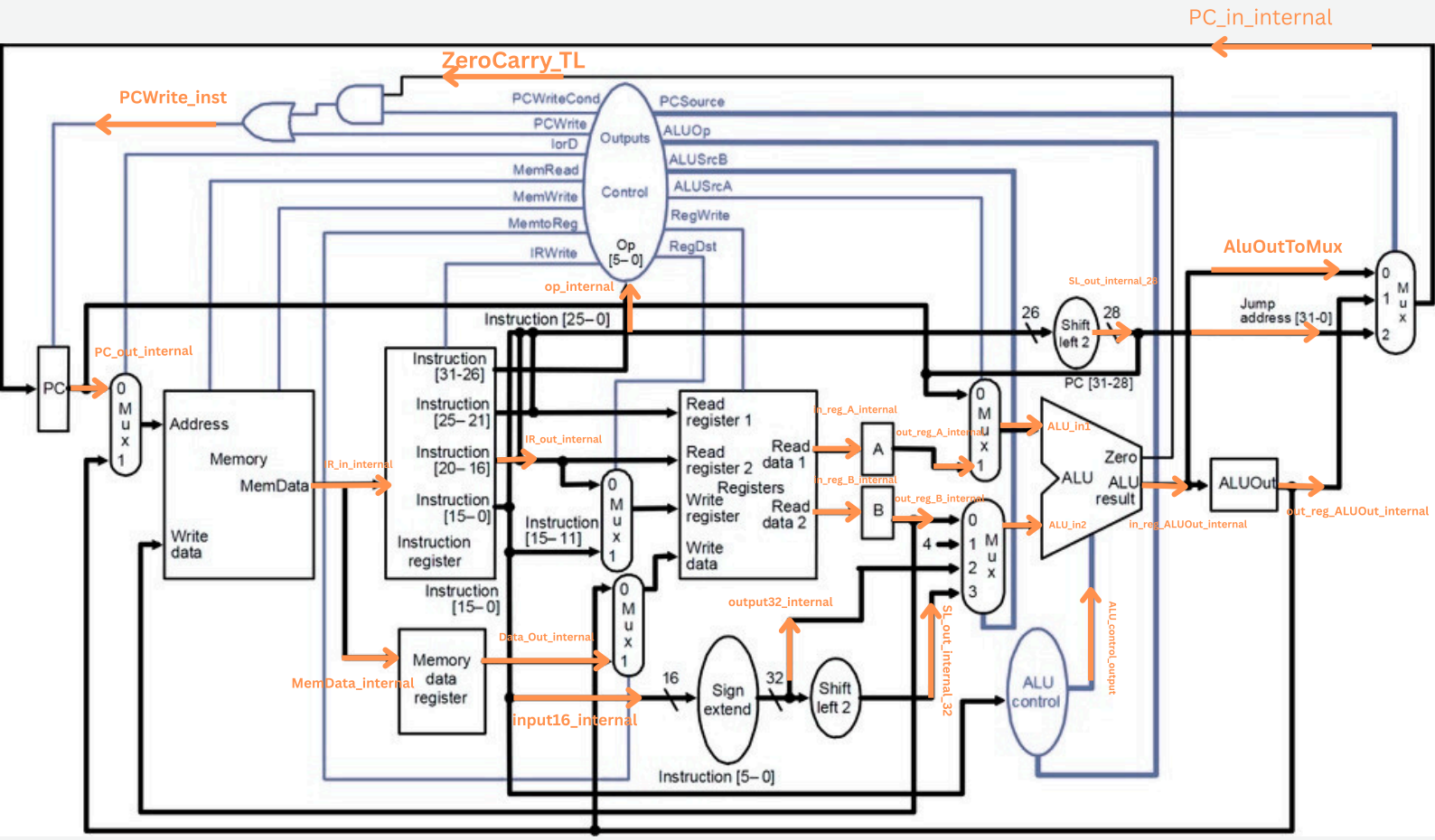
5 Simulation and Testing

6 Schematic Diagram

7 Inputs and Outputs

8 References

Overview of the Multi-Cycle Processor



The multi-cycle processor is a type of processor architecture that breaks down the execution of an instruction into multiple clock cycles.

Each step of the instruction—such as **fetch**, **decode**, **execute**, **memory access**, and **write-back**—is handled in a separate cycle.

This approach allows **reuse of functional units** (like the ALU and memory) across different steps, reducing hardware complexity and improving efficiency. The diagram above shows the datapath of a multi-cycle MIPS processor, including key components like the control unit, ALU, memory, and multiplexers.

DATAPATH

1. Program Counter (PC)

Holds the address of the next instruction to be executed.
Can be updated sequentially or through jump/branch instructions based on control signals.

2. Memory

Used for both instruction fetch and data access.
Has input for write data and output for memory data (MemData).
Controlled by the signals MemRead and MemWrite.

3. Instruction Register (IR)

Temporarily stores the fetched instruction from memory.
Extracts fields like opcode, rs, rt, rd, and immediate for further decoding.

4. Register File

Contains 32 general-purpose registers.
Uses rs and rt to read two source registers and rd or rt for the write-back destination.
Controlled by the RegWrite signal.

DATAPATH

5. Temporary Registers (A and B)

Store values read from the register file temporarily.
Serve as inputs to the ALU during execution stages.
Help separate datapath stages across multiple cycles.

6. Sign Extend Unit

Converts 16-bit immediate values to 32-bit values with proper sign extension.
Especially used for instructions like LW, SW, and BEQ.

7. Arithmetic Logic Unit (ALU)

Performs arithmetic and logical operations.
Controlled by ALUOp, ALUSrcA, and ALUSrcB signals.
Outputs result and a Zero flag (used for branch decisions).

8. ALU Control Unit

Determines the specific operation the ALU should perform (e.g., add, subtract, AND, OR).
Receives input from the main control unit and Function field (funct) of the instruction.

DATAPATH

9. Multiplexers (MUXes)

Select among multiple data sources at different stages:
ALU input source (register or immediate value).
Destination register (rd or rt).
Next PC value (sequential, branch, or jump address).
Controlled by various control lines (ALUSrcA, ALUSrcB, RegDst, PCSource).

10. Memory Data Register (MDR)

Temporarily holds data read from memory before it is written to a register.
Separates memory access from register write-back in different cycles.

Control Unit Signals

1. Multiplexer Select Signals

Multiplexer select signals determine the source of data at various decision points:

IorD: Selects whether the memory address comes from the PC or ALU output (used to choose between instruction fetch or data access).

MemtoReg: Selects whether to write back data from memory or ALU result to the register file.

RegDst: Selects whether to write the result to rt (for I-type instructions) or rd (for R-type).

ALUSrcA / ALUSrcB: Control which inputs go to the ALU (e.g., PC, register data, or sign-extended immediate).
PCSource: Determines the next PC value (e.g., ALU result, jump address, or ALUOut).

These signals dynamically switch sources at each phase of the instruction.

2. Register Enable Signals

These signals control register updates:

IRWrite: Enables writing to the Instruction Register during the fetch cycle.

RegWrite: Enables writing to the Register File during the write-back phase.

PCWrite / PCWriteCond: Allow the Program Counter (PC) to update, either unconditionally or based on a branch condition (e.g., zero flag).

Each of these ensures that only the correct register is updated in the correct cycle.

3. Memory Access Signals

These control how the memory block is accessed:

MemRead: Enables reading from memory (used during instruction fetch or load operations).

MemWrite: Enables writing to memory (used in store instructions).

These are essential for distinguishing between instruction/data fetch and store operations.

4. ALU Control Signals

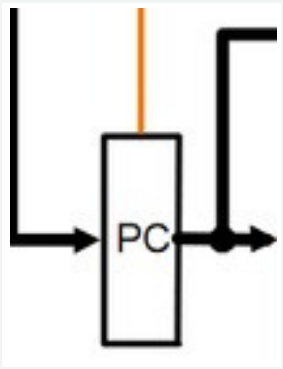
These signals determine the specific operation performed by the ALU:

ALUOp: Provides a general classification of the instruction (e.g., load/store, branch, R-type) and is used as input to the ALU Control Unit.

ALU Control Unit: Uses ALUOp along with the function field of the instruction (for R-type) to generate the exact ALU operation.

These signals are critical for executing arithmetic and logic instructions correctly across different stages in the multi- cycle datapath.

PC




```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity PC is
 port(
 PC_in     : in std_logic_vector(31 downto 0)  ;
 PCWrite   : in std_logic ;

   clk      : in  std_logic;
   reset_neg : in  std_logic;
 PC_out    : out std_logic_vector(31 downto 0)
 );

end PC;

architecture Behavioral of PC is
begin

   process(clk, reset_neg)
   begin
      if reset_neg = '0' then
         PC_out <= (others => '0');
      elsif rising_edge(clk) and PCWrite = '1'  then
         PC_out <= PC_in;
      end if;
 end process;

end Behavioral;
```

MUX 2in 32-bit

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity mux_2in_32bit is
 port(
 --input
 input_1   : in std_logic_vector(31 downto 0);
 input_2   : in std_logic_vector(31 downto 0);
 mux_select: in std_logic;

 --output
 output    : out std_logic_vector(31 downto 0)
 );
end mux_2in_32bit;

architecture Behavioral of mux_2in_32bit is
begin
 with mux_select select
 output <= input_1 when '0',
         input_2 when others;
end Behavioral;
```

Memory

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity Memory is
Port(
-- INPUTS
   Clock    : in std_logic;
   reset_neg : in std_logic;
  Address : in std_logic_vector(31 downto 0);  MemWrite : in std_logic;  MemRead : in std_logic;
 WriteData : in std_logic_vector(31 downto 0);  -- OUTPUTS  MemData : out std_logic_vector(31
 downto 0)  ); end Memory; architecture rtl of Memory is  type mem_type is array (0 to 63) of
 std_logic_vector(7 downto 0); -- Memory size set to 64 bytes  signal mem : mem_type := (  --
 Initializing memory with some values for simulation or testing  0 => "10001110", -- lw $R0,
 47($R20)
 1 => "10000000",
 2 => "00000000",
 3 => "00101111",
 4 => "00100000", -- addi $R1, $R3, 50
 5 => "01100001",
 6 => "00000000",
 7 => "00110010",
 8 => "00100000", -- addi $R2, $R3, 48
 9 => "01100010",
 10 => "00000000",
 11 => "00110000",
 12 => "00000000", -- add $R2, $R2, $R0
 13 => "01000000",
 14 => "00010000",
 15 => "00100000",
 16 => "00010000", -- beq $R1, $R2, 1
 17 => "00100010",
 18 => "00000000",
 19 => "00000001",
 20 => "00001000", -- j 3
 21 => "00000000",
 22 => "00000000",
 23 => "00000011",
 24 => "00000000", -- add $R0, $R1, $R2
 25 => "00100010",
 26 => "00000000",
 27 => "00100000",
 28 => "10001110", -- lw $R10, 47($R20)
 29 => "10001010",
 30 => "00000000",
 31 => "00101111",

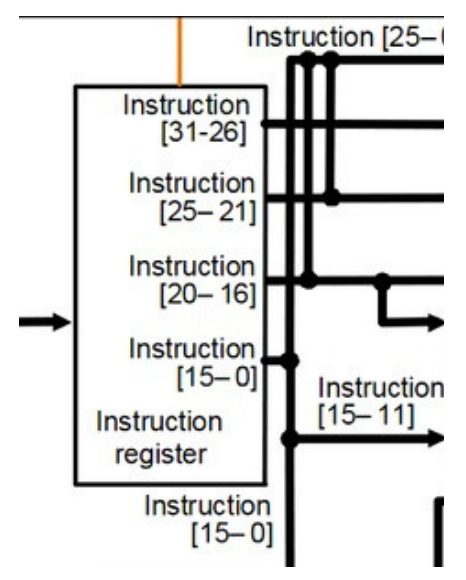
 50 => "00000001",
 );
 others => "00000000" -- Filling the rest with zeroes  );
```

```
begin
 process(Clock,reset_neg)
 begin
  if rising_edge(Clock) and MemWrite='1' then
        mem(to_integer(unsigned(address)))     <= WriteData(31 downto 24);
        mem(to_integer(unsigned(address) + 1)) <= WriteData(23 downto 16);
        mem(to_integer(unsigned(address) + 2)) <= WriteData(15 downto 8);
        mem(to_integer(unsigned(address) + 3)) <= WriteData(7  downto 0);
    end if;
 end process;

 MemData <= mem(to_integer(unsigned(Address))) &
          mem(to_integer(unsigned(Address) + 1)) &
          mem(to_integer(unsigned(Address) + 2)) &
          mem(to_integer(unsigned(Address) + 3))
   when MemRead = '1' and to_integer(unsigned(Address)) <= 52
   else (others => '0');

end behavior;
```

Instruction Register

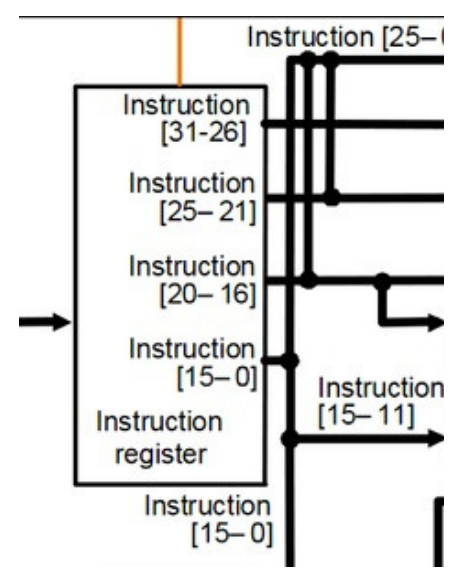



```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity Instruction_Register is
port(
 IR_in : in std_logic_vector(31 downto 0);
 IRWrite : in std_logic;
clk : in std_logic;
reset : in std_logic;
IR_out : out std_logic_vector(31 downto 0)
);
end Instruction_Register;

architecture Behavioral of Instruction_Register is
begin
  process(clk, reset)
    begin
      if reset = '1' then
         IR_out <= (others => '0');
      elsif rising_edge(clk) and IRWrite = '1' then
         IR_out <= IR_in;
      end if;
 end process;
end Behavioral;
```

Instruction Register




```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity Instruction_Register is
 port(
 IR_in       : in  std_logic_vector(31 downto 0);
 IRWrite   : in  std_logic;

 clk      : in  std_logic;
   reset_neg : in  std_logic;

 IR_out    : out  std_logic_vector(31 downto 0)
 );

end Instruction_Register;

architecture Behavioral of Instruction_Register is
begin

  process(clk, reset_neg)
    begin
      if reset_neg = '0' then
         IR_out <= (others => '0');
      elsif rising_edge(clk) and IRWrite = '1'  then
         IR_out <= IR_in;
      end if;
 end process;


end Behavioral;
```

Memory Data Register

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity MDR is
 Port(
 --INPUTS
 Clock     : in std_logic;
 Data_In   : in std_logic_vector(31 downto 0);
 reset_neg : in std_logic;
 --OUTPUTS
 Data_Out  : out std_logic_vector(31 downto 0)
 );
end MDR;

architecture behavior of MDR is
  type mem_data_type is array (0 downto 0) of std_logic_vector(31 downto 0);
  signal MemDataReg: mem_data_type := ((others => (others => '0')));
Begin
 process(Clock)
 begin
 if reset_neg = '0' then -- reset
         MemDataReg(0) <= (others => '0');
 elsif rising_edge(Clock) then
    MemDataReg(0) <= Data_In;
    end if;
 end process;


 Data_Out <= MemDataReg(0);

end behavior;
```

Register File

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Registers is
  port(
  --INPUTS
  clk         : in std_logic;
  reset_neg    : in std_logic;
  reg_write    : in std_logic;
  read_reg1    : in std_logic_vector(4 downto 0);
  read_reg2    : in std_logic_vector(4 downto 0);
  write_reg    : in std_logic_vector(4 downto 0);
  write_data   : in std_logic_vector(31 downto 0);
  --OUTPUTS
  read_data1   : out std_logic_vector(31 downto 0);
  read_data2   : out std_logic_vector(31 downto 0)
 );
end Registers;

architecture behavior of Registers is

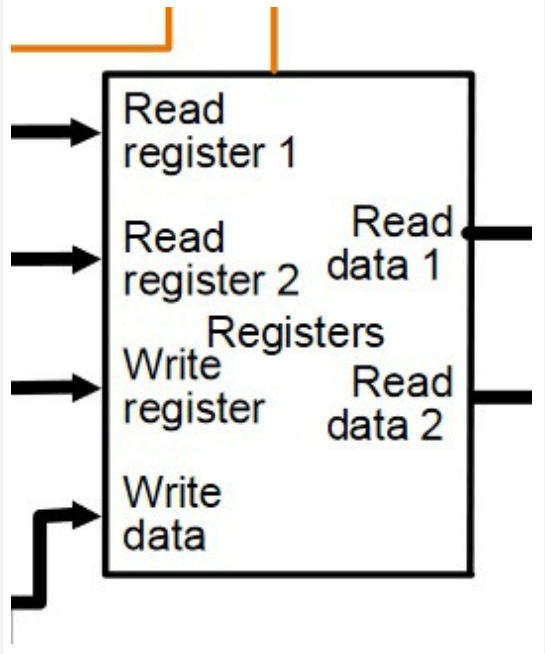
type register_type is array (0 to 31) of std_logic_vector(31 downto 0);
signal reg : register_type := ((others => (others => '0')));

begin

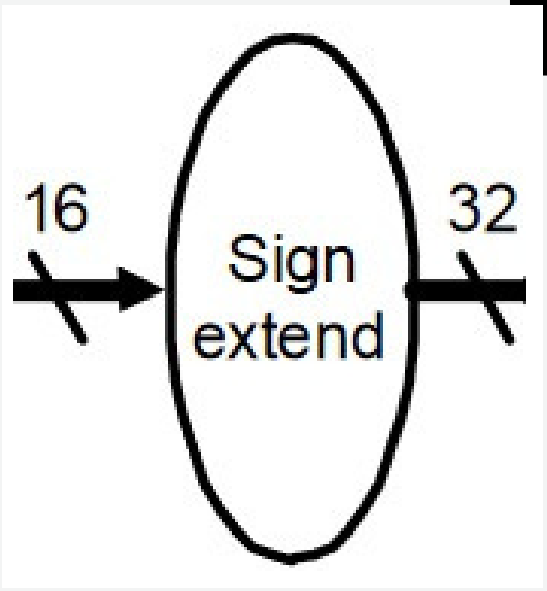
  process(read_reg1,read_reg2,reg)
  begin
   read_data1 <= reg(to_integer(unsigned(read_reg1)));
   read_data2 <= reg(to_integer(unsigned(read_reg2)));
 end process;

 process(clk)
 begin
   if reset_neg = '0' then -- reset
        reg(to_integer(unsigned(write_reg))) <= (others => '0');
   elsif rising_edge(clk) and reg_write='1' and write_reg /="00000" then
        reg(to_integer(unsigned(write_reg))) <= write_data;
   end if;

 end process;
 end behavior;
```



Sign Extend

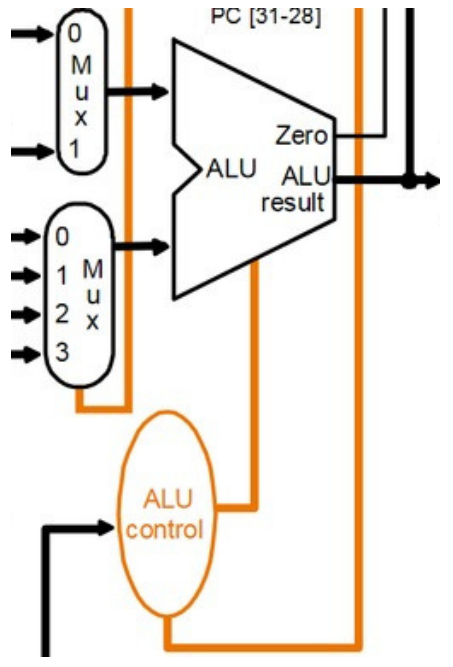



```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity SignExtend is
   port (
      input16  : in  std_logic_vector(15 downto 0);
      output32 : out std_logic_vector(31 downto 0)
   );
end entity;

architecture rtl of SignExtend is
begin
   output32 <= "0000000000000000" & input16 when
(input16(15) = '0') else
               "1111111111111111" & input16;
end architecture;
```

ALU




```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity ALU is
 Generic(n: integer:= 32);
 port(
 --input
 operand_1  : in std_logic_vector(31 downto 0);
 operand_2  : in std_logic_vector(31 downto 0);
 ALU_CONTROL: in std_logic_vector(3  downto 0);

 --output
 result: out std_logic_vector(31 downto 0);
 zero  : out std_logic
 );
end ALU;

architecture Behavioral of ALU is

signal temp: std_logic_vector(31 downto 0);
begin

 temp <=
  --add
  std_logic_vector(unsigned(operand_1) + unsigned(operand_2)) when ALU_CONTROL = "0000" else
  --subtract
  std_logic_vector(unsigned(operand_1) - unsigned(operand_2)) when ALU_CONTROL = "0001" else
  --AND
  operand_1 AND operand_2 when ALU_CONTROL = "0010" else
  --OR
  operand_1 OR  operand_2 when ALU_CONTROL = "0011" else
  --NOR
  operand_1 NOR operand_2 when ALU_CONTROL = "0100" else
  --XOR
  operand_1 XOR operand_2 when ALU_CONTROL = "0101" else
  --shift left logical
  std_logic_vector(shift_left (unsigned(operand_1), to_integer(unsigned(operand_2(10 downto 6))))) when ALU_CONTROL = "0111" else
    --shift right logical
  std_logic_vector(shift_right(unsigned(operand_1), to_integer(unsigned(operand_2(10 downto 6))))) when ALU_CONTROL = "1000" else
       --in other cases
  (others => '0');

  zero <= '1' when temp <= "00000000000000000000000000000000" else '0';
   result <= temp;

end Behavioral;
```

ALU Control

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity ALU_CONTROL is
 port(
 --input
 ALUOp : in std_logic_vector(1 downto 0);
 instr : in std_logic_vector(5 downto 0);

 --output
 result: out std_logic_vector(3 downto 0)
 );
end ALU_CONTROL;

architecture Behavioral of ALU_CONTROL is

signal temp, operation: std_logic_vector(3 downto 0) := "1111";
begin
 with ALUOp select
 temp <= "0000"    when "00", -- addi for LW
        "0110"    when "01", -- subi for Branch
        operation when "10", -- R-type
        "1111"    when others; -- in other cases

 with instr select
 operation <= "0000" when "100000", --add
    "0001" when "100010", --Substract
    "0010" when "100100", --AND
    "0011" when "100101", --OR
    "0100" when "100111", --NOR
    "0101" when "100110", --XOR
    "1001" when "001000", --add immediate
    "1111" when others;
 result <= temp;

end Behavioral;
```

Shift Left

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

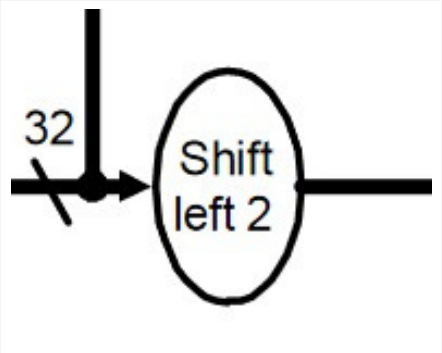
entity Shift_Left32To32 is
 port(
 SL_in  : in std_logic_vector(31 downto 0);

 SL_out : out std_logic_vector(31 downto 0)
 );
end Shift_Left32To32;

architecture Behavioral of Shift_Left32To32 is
begin

 SL_out <= SL_in(29 downto 0) & "00" ;

 end Behavioral;
```



```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

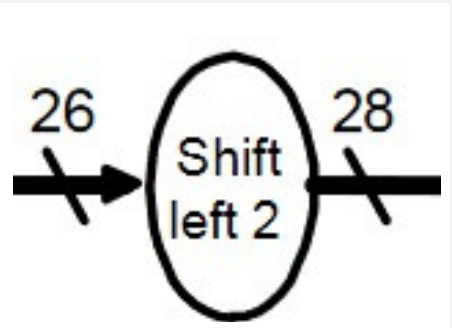
entity Shift_Left26To28 is
 port(
 SL_in  : in std_logic_vector(25 downto 0);

 SL_out : out std_logic_vector(27 downto 0)
 );
end Shift_Left26To28;

architecture Behavioral of Shift_Left26To28 is
begin

 SL_out <= SL_in & "00" ;

 end Behavioral;
```



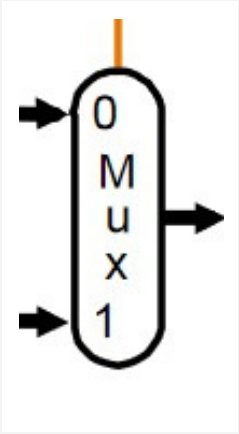
MUXs

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity mux_2in_5bit is
 port(
 --input
 input_1   : in std_logic_vector(4 downto 0);
 input_2   : in std_logic_vector(4 downto 0);
 mux_select: in std_logic;

 --output
 output    : out std_logic_vector(4 downto 0)
 );
end mux_2in_5bit;

architecture Behavioral of mux_2in_5bit is
begin
 with mux_select select
 output <= input_1 when '0',
         input_2 when others;
end Behavioral;
```

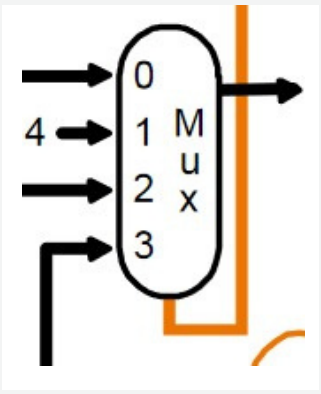


```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity mux_4in_32bit is
 port(
 --input
 input_1   : in std_logic_vector(31 downto 0);
 input_2   : in std_logic_vector(31 downto 0);
 input_3   : in std_logic_vector(31 downto 0);
 input_4   : in std_logic_vector(31 downto 0);
 mux_select: in std_logic_vector(1 downto 0);

 --output
 output    : out std_logic_vector(31 downto 0)
 );
end mux_4in_32bit;

architecture Behavioral of mux_4in_32bit is
signal mux_out: std_logic_vector(31 downto 0);
begin
 with mux_select select
 mux_out <= input_1 when "00",
         input_2 when "01",
    input_3 when "10",
    input_4 when "11",
    (others => '0') when others;
 output <= mux_out;
end Behavioral;
```

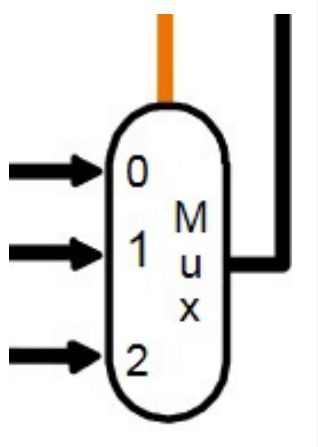


```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

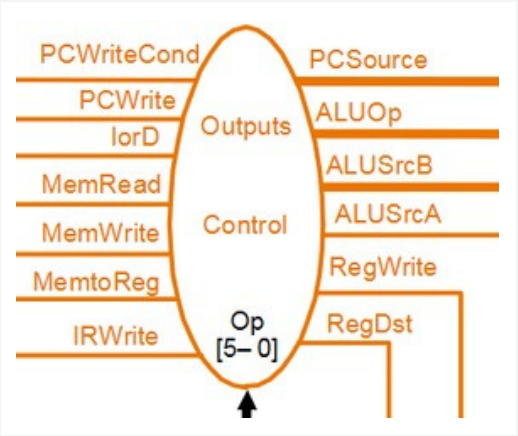
entity mux_3in_32bit is
 port(
 --input
 input_1   : in std_logic_vector(31 downto 0);
 input_2   : in std_logic_vector(31 downto 0);
 input_3   : in std_logic_vector(31 downto 0);
 mux_select: in std_logic_vector(1 downto 0);

 --output
 output    : out std_logic_vector(31 downto 0)
 );
end mux_3in_32bit;

architecture Behavioral of mux_3in_32bit is
signal mux_out: std_logic_vector(31 downto 0);
begin
 with mux_select select
 mux_out <= input_1 when "00",
          input_2 when "01",
    input_3 when "10",
    (others => '0') when others;
 output <= mux_out;
end Behavioral;
```



Control Unit




```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity ControlUnit is
   port (
      -- Inputs
      op          : in  std_logic_vector(5 downto 0);
      CLK         : in  std_logic;
      Reset       : in  std_logic;

      -- Outputs
      RegDst      : out std_logic;
      ALUSrcA     : out std_logic;
      ALUSrcB     : out std_logic_vector(1 downto 0);
      MemRead     : out std_logic;
      MemWrite    : out std_logic;
      MemtoReg    : out std_logic;
      RegWrite    : out std_logic;
      Branch      : out std_logic;
      Jump        : out std_logic;
      ALUOp       : out std_logic_vector(1 downto 0);
      PCWrite     : out std_logic;
      PCWriteCond : out std_logic;
      IorD        : out std_logic;
      IRWrite     : out std_logic;
      PCSource    : out std_logic_vector(1 downto 0)
   );
end entity;

architecture rtl of ControlUnit is

  type state is( InstructionFetch,
                 InstructionDecode,
                 MemoryAddressComp,
                 Execution,
                 Execution_I,
                 BranchCompletion,
                 JumpCompletion,
                 MemoryAccessLoad,
                 MemoryAccessStore,
                 RTypeCompletion,
                 RTypeCompletion_I,
                 MemoryReadCompletionStep );

  signal current_state, next_state : state;
  signal ctrl_state : std_logic_vector(15 downto 0) := (others =>
'0');
```

```
begin

process(CLK, Reset, Op)
begin
if Reset = '0' then
current_state <= InstructionFetch;
elsif rising_edge(CLK) then
current_state <= next_state;
end if;

case current_state is

when InstructionFetch => next_state <= InstructionDecode;

when InstructionDecode => if Op = "100011" then -- lw
next_state <= MemoryAddressComp;
elsif Op = "101011" then -- sw
next_state <= MemoryAddressComp;
elsif Op = "000000" then -- R-type
next_state <= Execution;
elsif Op = "001000" then -- immediate
next_state <= Execution_I;
elsif Op = "000100" then -- BEQ
next_state <= BranchCompletion;
elsif Op = "000010" then -- Jump
next_state <= JumpCompletion;
end if;

when MemoryAddressComp => if Op = "100011" then -- lw
next_state <= MemoryAccessLoad;
else -- sw
next_state <= MemoryAccessStore;
end if;

when Execution => next_state <= RTypeCompletion;
when Execution_I => next_state <= RTypeCompletion_I;
when MemoryAccessLoad => next_state <= MemoryReadCompletionStep;
when MemoryAccessStore => next_state <= InstructionFetch;
when BranchCompletion => next_state <= InstructionFetch;
when JumpCompletion => next_state <= InstructionFetch;
when RTypeCompletion => next_state <= InstructionFetch;
when others => next_state <= InstructionFetch;

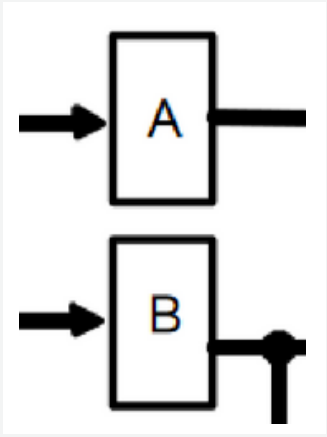
end case;
end process;
```

```
with current_state select
ctrl_state <= "1001001000001000" when InstructionFetch,
"0000000000011000" when InstructionDecode,
"0000000000010100" when MemoryAddressComp,
"0000000001000100" when Execution,
"0000000000010100" when Execution_I,
"0100000010100100" when BranchCompletion,
"1000000100000000" when JumpCompletion,
"0011000000000000" when MemoryAccessLoad,
"0010100000000000" when MemoryAccessStore,
"0000000000000011" when RTypeCompletion,
"0000000000000010" when RTypeCompletion_I,
"0000010000000010" when MemoryReadCompletionStep,
"0000000000000000" when others;

PCWrite <= ctrl_state(15);
PCWriteCond <= ctrl_state(14);
IorD <= ctrl_state(13);
MemRead <= ctrl_state(12);
MemWrite <= ctrl_state(11);
MemToReg <= ctrl_state(10);
IRWrite <= ctrl_state(9);
PCSource <= ctrl_state(8 downto 7);
ALUOp <= ctrl_state(6 downto 5);
ALUSrcB <= ctrl_state(4 downto 3);
ALUSrcA <= ctrl_state(2);
RegWrite <= ctrl_state(1);
RegDst <= ctrl_state(0);

end architecture ;
```

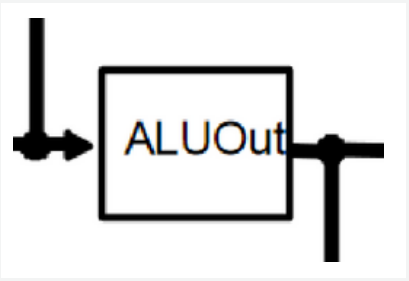
TempRegisters




```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity TempRegisters is
  port( -- input
        CLK        : in std_logic;
        reset_neg   : in std_logic;
        in_reg_A    : in std_logic_vector (31 downto 0);
        in_reg_B    : in std_logic_vector (31 downto 0);
        in_ALU_out  : in std_logic_vector (31 downto 0);

        -- output
        out_reg_A   : out std_logic_vector(31 downto 0);
        out_reg_B   : out std_logic_vector(31 downto 0);
        out_ALU_out : out std_logic_vector(31 downto 0) );
end TempRegisters;
```




```
architecture Behavioral of TempRegisters is
  type registers is array (0 to 2) of std_logic_vector(31 downto 0);
  signal data : registers := ((others => (others => '0')));

begin
  process(CLK)
  begin
    if reset_neg = '0' then -- reset
      data <= ((others => (others => '0')));
    else if rising_edge(CLK) then
      data(0) <= in_reg_A;
      data(1) <= in_reg_B;
      data(2) <= in_ALU_out;
    end if;
    end if;
  end process;

  out_reg_A   <= data(0);
  out_reg_B   <= data(1);
  out_ALU_out <= data(2);

end  Behavioral;
```

Full Code

```
library ieee;
use ieee.std_logic_1164.all;

entity multicycle is
  port(
    CLK       : in std_logic;
    reset_neg  : in std_logic
 );
end multicycle;

architecture Behavioral of multicycle is

constant PC_increment : std_logic_vector(31 downto 0) :=
"00000000000000000000000000000100";

-- 32-bit std_logic_vectors
signal AluOutToMux, ALUOut, ALU_in1, ALU_in2, read_data1_internal, PC_in_internal,
 read_data2_internal, Jump_address, SL_out_internal_32, output32_internal,
 Write_data_internal, Data_Out_internal, PC_out_internal, address_internal: std_logic_vector(31
downto 0);

-- 27-bit vector for jump shift
signal SL_out_internal_28: std_logic_vector(27 downto 0);

-- 5-bit signal for write register
signal Write_register_internal: std_logic_vector(4 downto 0);

-- 4-bit ALU control
signal ALU_control_output: std_logic_vector(3 downto 0);


-- 2-bit control signals
signal ALUOp_internal, PCSource_internal, ALUSrcB_internal: std_logic_vector(1 downto 0);

-- Single-bit control signals
signal PCWrite_inst, ZeroCarry_TL, RegDst_internal, ALUSrcA_internal, MemRead_internal,
IRWrite_internal,
 MemWrite_internal, MemtoReg_internal, RegWrite_internal, Branch_internal,
 Jump_internal, PCWrite_internal, PCWriteCond_internal, IorD_internal: std_logic;
```

```
-- Register signals
signal IR_out_internal, in_reg_A_internal, in_reg_B_internal, in_reg_ALUOut_internal,
 out_reg_A_internal, out_reg_B_internal: std_logic_vector(31 downto 0);

begin

PCWrite_inst <= PCWrite_internal or ( PCWriteCond_internal and ZeroCarry_TL );
Jump_address(31 downto 28) <= PC_out_internal(31 downto 28);

-- Program Counter
PC_inst: entity work.PC
 port map (
 PC_in => PC_in_internal,
 PCWrite => PCWrite_inst,
 CLK => CLK,
 reset_neg => reset_neg,
 PC_out => PC_out_internal
 );

-- MUX between PC and Memory
MUX1_2in_32bit: entity work.mux_2in_32bit
 port map(
 input_1 => PC_out_internal,
 input_2 => AluOutToMux,
 mux_select => IorD_internal,
 output => address_internal
 );

-- MEMORY
MEMORY: entity work.Memory
 port map(
  Address   => address_internal,
  MemWrite  => MemWrite_internal,
       MemRead   => MemRead_internal,
       WriteData => out_reg_B_internal,
       MemData   => Data_Out_internal,
  Clock     => CLK,
    reset_neg => reset_neg
 );

-- Instruction Register
INSTRUCTION_REGISTER: entity work.Instruction_Register
 port map(
  IR_in     => Data_Out_internal,
  IRWrite   => IRWrite_internal,
  IR_out    => IR_out_internal,
  clk       => CLK,
    reset_neg => reset_neg
 );
```

```
-- MDR
MDR: entity work.MDR
 port map(
 Data_In => Data_Out_internal,
 Data_Out => Data_Out_internal,
 Clock => CLK,
 reset_neg => reset_neg
 );

-- Control unit
Control_unit: entity work.ControlUnit
 port map (
   op         => IR_out_internal(31 downto 26),
   RegDst     => RegDst_internal,
   ALUSrcA    => ALUSrcA_internal,
   ALUSrcB    => ALUSrcB_internal,
   MemRead    => MemRead_internal,
   MemWrite   => MemWrite_internal,
   MemtoReg   => MemtoReg_internal,
   RegWrite   => RegWrite_internal,
   Branch     => Branch_internal,
   Jump       => Jump_internal,
   ALUOp      => ALUOp_internal,
   PCWrite    => PCWrite_internal,
   PCWriteCond => PCWriteCond_internal,
   IorD       => IorD_internal,
   IRWrite    => IRWrite_internal,
   PCSource   => PCSource_internal,
   CLK        => CLK,
   Reset      => reset_neg
 );

-- MUX: Write register
MUX_2in_5bit: entity work.mux_2in_5bit
 port map(
  input_1     => IR_out_internal(20 downto 16),
  input_2     => IR_out_internal(15 downto 11),
  mux_select  => RegDst_internal,
  output   => Write_register_internal
 );

-- MUX: Memory to Register
MUX2_2in_32bit: entity work.mux_2in_32bit
 port map(
  input_1    => AluOutToMux,
  input_2    => Data_Out_internal,
  mux_select  => MemtoReg_internal,
  output    => Write_data_internal
 );
```

```
-- Sign Extend
SignExtend: entity work.SignExtend
 port map (
 input16 => IR_out_internal(15 downto 0),
 output32 => output32_internal
 );

-- Shift Left 32 to 32
ShiftLeft1: entity work.Shift_Left32To32
 port map(
  SL_in  => output32_internal,
  SL_out => SL_out_internal_32
 );

-- Shift Left 26 to 28
ShiftLeft2: entity work.Shift_Left26To28
 port map(
  SL_in  => IR_out_internal(25 downto 0),
  SL_out => Jump_address(27 downto 0)
 );

-- ALU Control
ALU_CONTROL: entity work.ALU_CONTROL
 port map(
  ALUOp  => ALUOp_internal,
  instr  => IR_out_internal(5 downto 0),
    result => ALU_control_output
 );

-- Registers
Registers: entity work.Registers
 port map(
  clk       => CLK,
  reset_neg  => reset_neg,
  reg_write  => RegWrite_internal,
  read_reg1  => IR_out_internal(25 downto 21),
  read_reg2  => IR_out_internal(20 downto 16),
  write_reg  => Write_register_internal,
  write_data => Write_data_internal,
  read_data1 => read_data1_internal,
  read_data2 => read_data2_internal
 );

-- ALU
ALU: entity work.ALU
 port map(
  operand_1   => ALU_in1,
  operand_2   => ALU_in2,
  ALU_CONTROL => ALU_control_output,
  result      => ALUOut,
  zero        => ZeroCarry_TL
 );
```

```
-- MUX for ALU operand 1
MUX3_2in_32bit: entity work.mux_2in_32bit
 port map(
 input_1 => PC_out_internal,
 input_2 => out_reg_A_internal,
 mux_select => ALUSrcA_internal,
 output => ALU_in1
 );

-- MUX for ALU operand 2
MUX3_4in_32bit: entity work.mux_4in_32bit
 port map(
 input_1 => out_reg_B_internal,
 input_2 => PC_increment,
 input_3 => output32_internal,
 input_4 => SL_out_internal_32,
 output => ALU_in2,
 mux_select => ALUSrcB_internal
 );

-- Last MUX: PC source
MUX_3in_32bit: entity work.mux_3in_32bit
 port map(
  input_1    => ALUOut,
  input_2    => AluOutToMux,
  input_3    => Jump_address,
  output     => PC_in_internal,
  mux_select => PCSource_internal
 );

-- Temp Registers
TempRegisters: entity work.TempRegisters
 port map(
  CLK        => CLK,
       reset_neg   => reset_neg,
       in_reg_A    => read_data1_internal,
       in_reg_B    => read_data2_internal,
       in_ALU_out  => ALUOut,
       out_reg_A   => out_reg_A_internal,
       out_reg_B   => out_reg_B_internal,
       out_ALU_out => AluOutToMux
 );
end Behavioral;
```

tb_full_code

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use std.textio.all;

entity TestBench is
end TestBench;

architecture Behavioral of TestBench is
  -- constants
  constant CLK_low    : time := 12 ns;
  constant CLK_high   : time := 8 ns;
  constant CLK_period : time := CLK_low + CLK_high;
  constant ResetTime  : time := 10 ns;
  -- dut signals
  signal clock , reset : std_logic;

component multicycle is
  port( CLK, reset_neg  : in std_logic );
end component;

begin
  dut : multicycle port map ( clock, reset );

  -- reset
  reset_process : process
```

tb_full_code

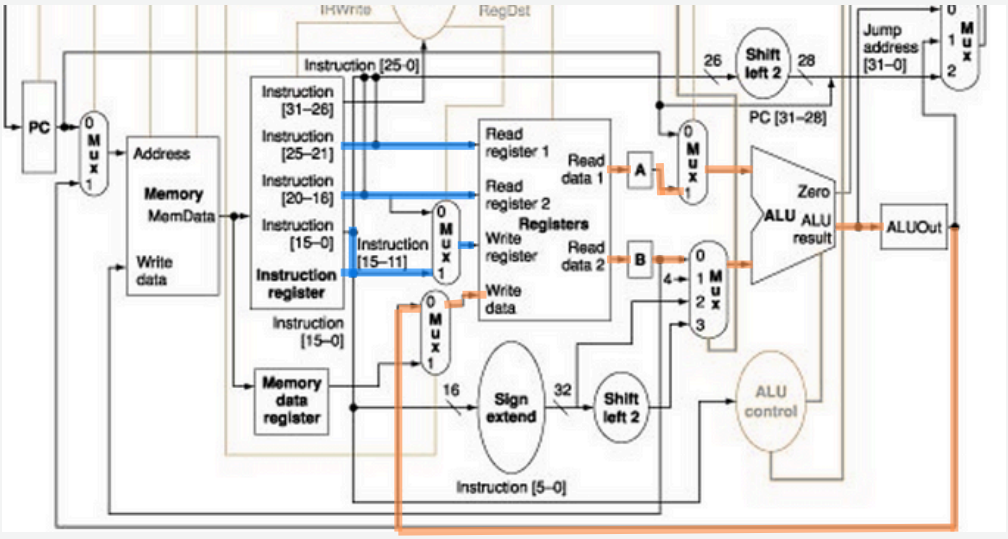
```
begin
reset <= '0';
wait for ResetTime;
reset <= '1';
wait;
end process reset_process;

clock_process : process

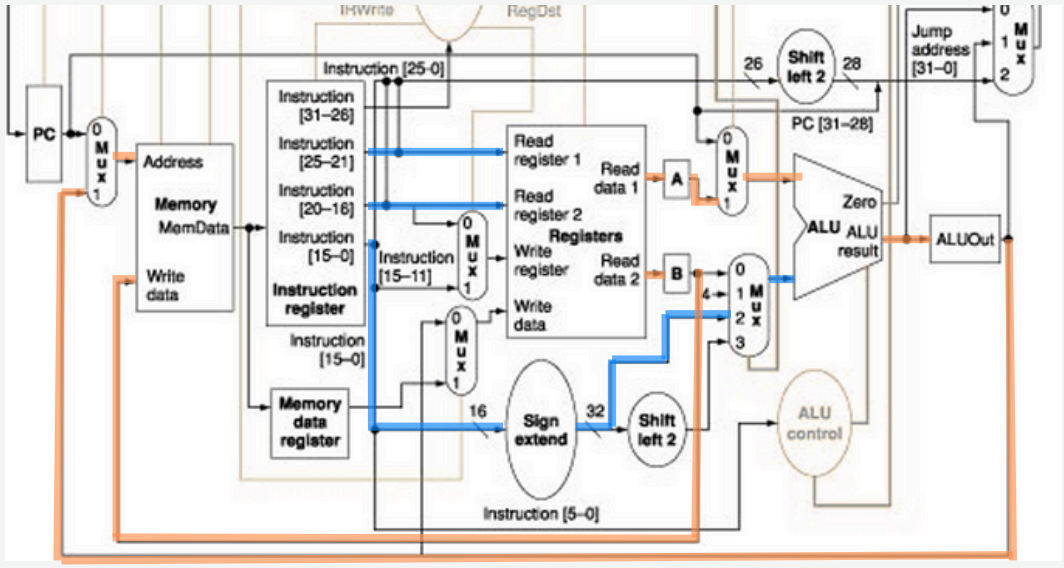
begin
if (clock = '0') then
clock <= '1';
wait for CLK_high;
else
clock <= '0';
wait for CLK_low;
end if;
end process clock_process;

end Behavioral;
```

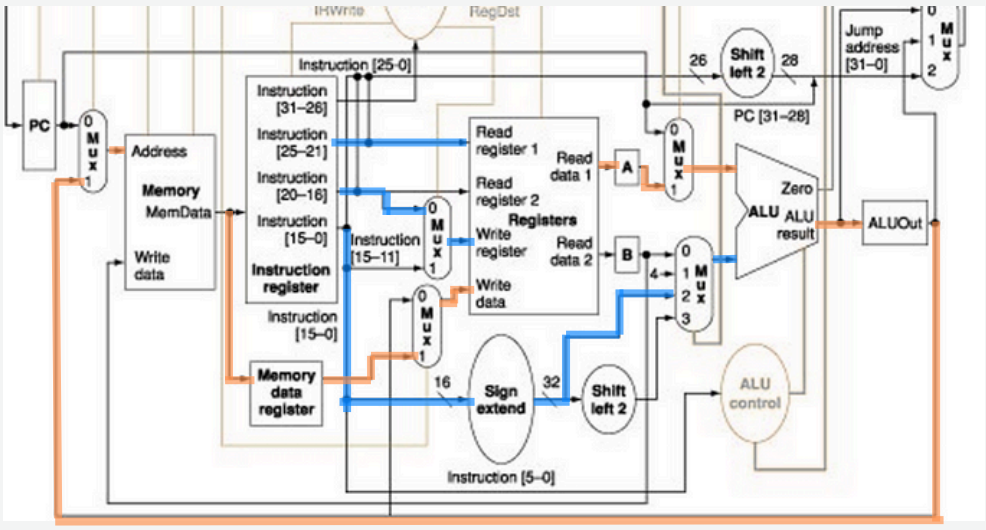
R-Type Instruction



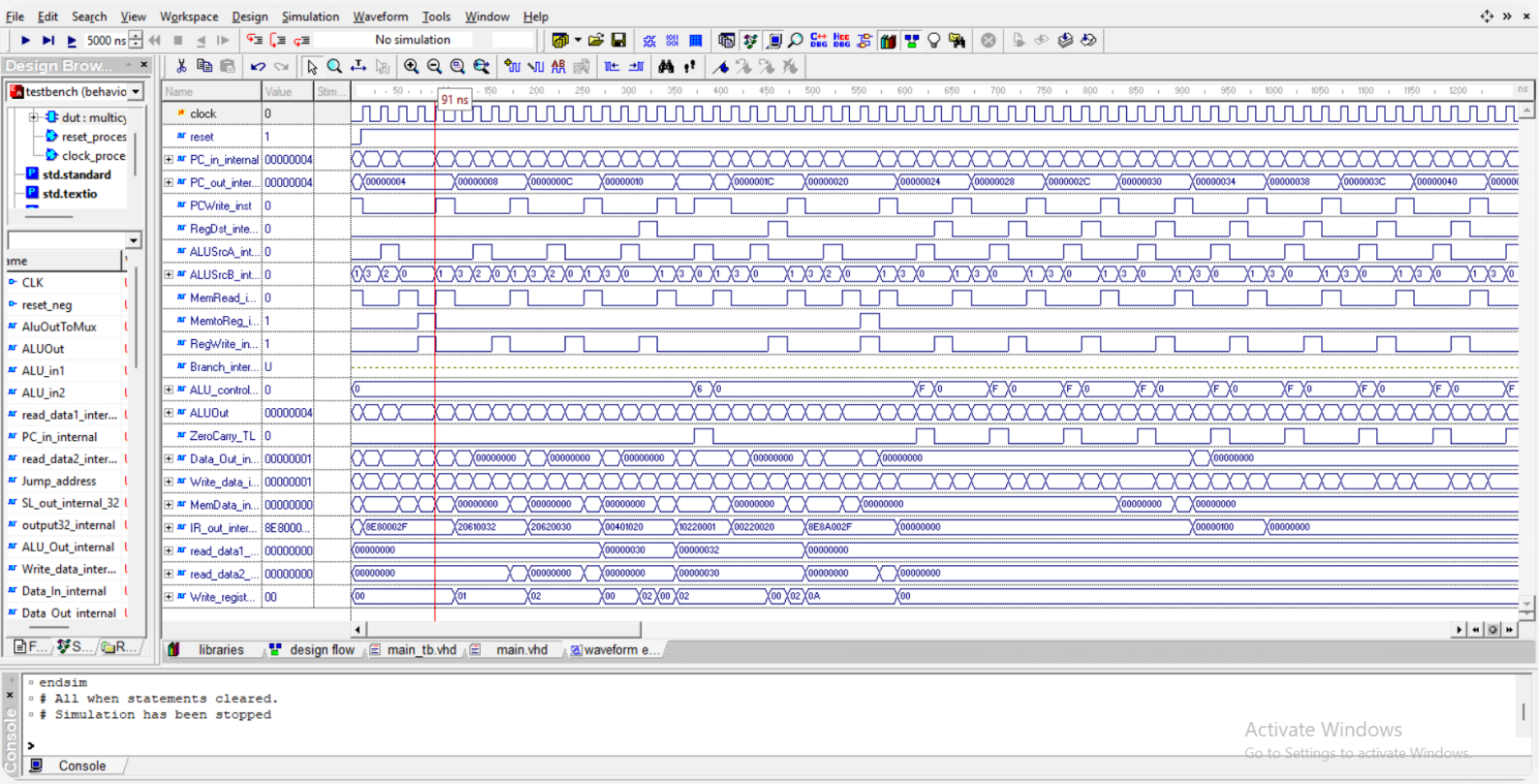
Store Instruction



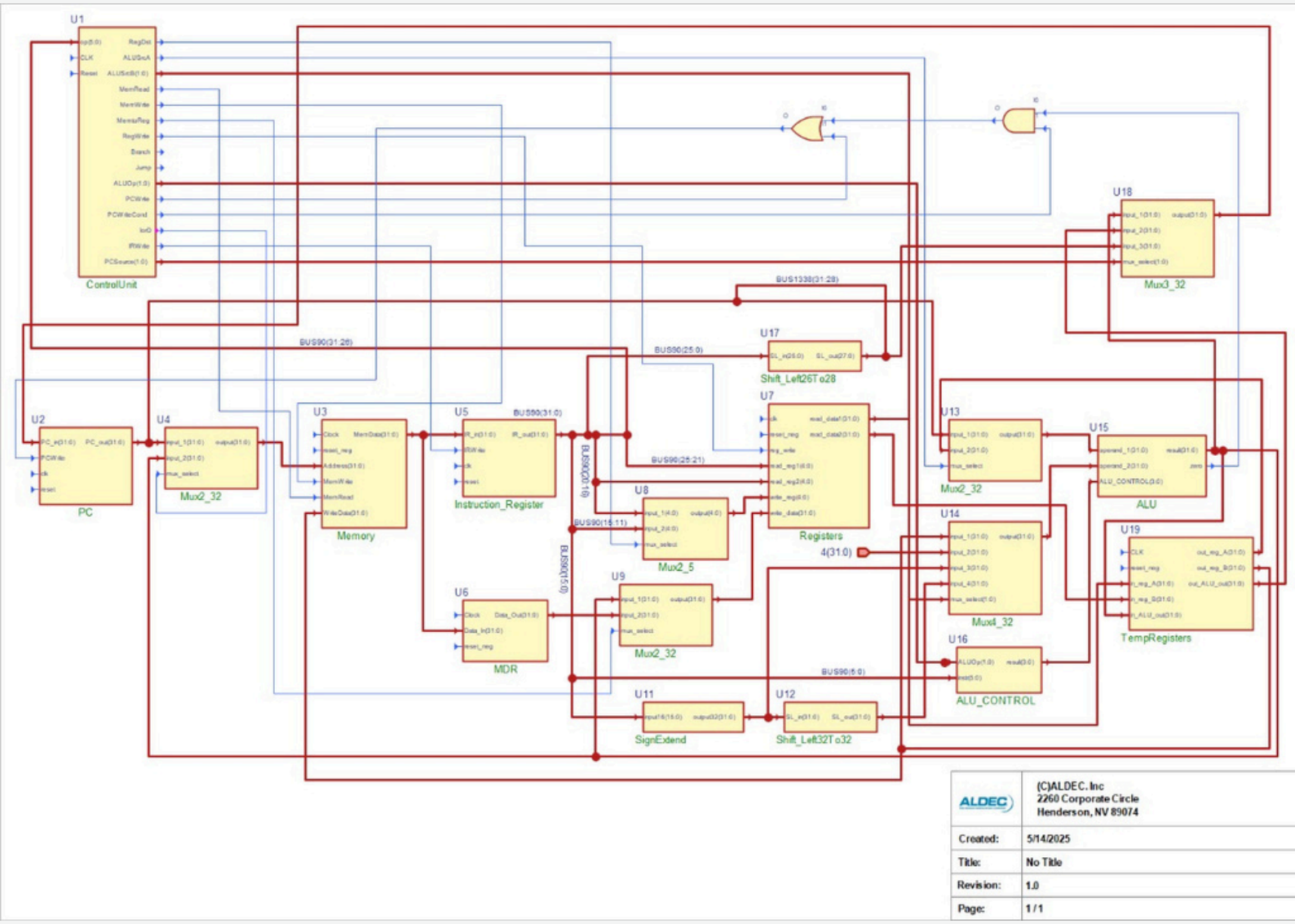
Load Instruction



Run in Active Hdl

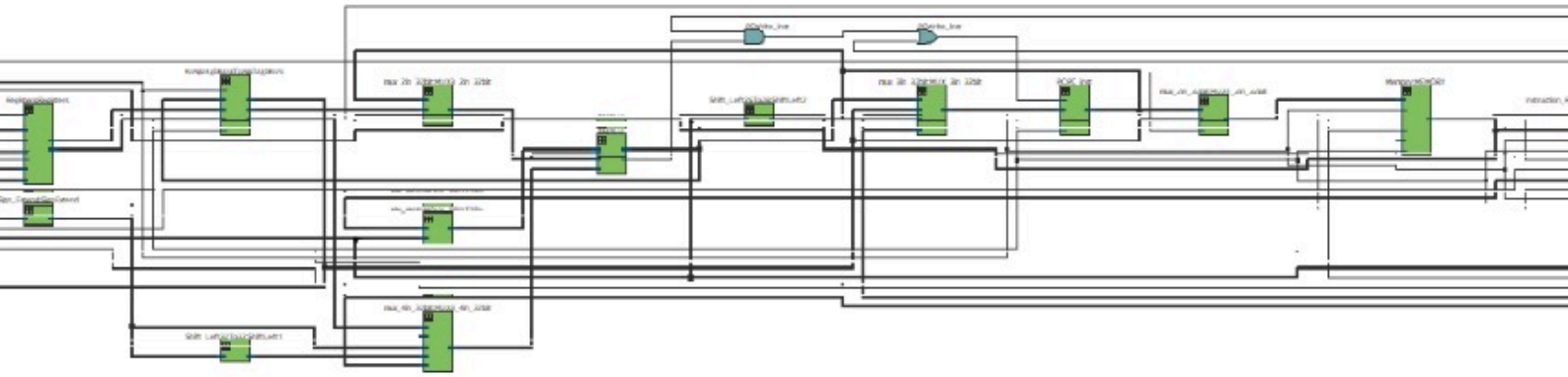


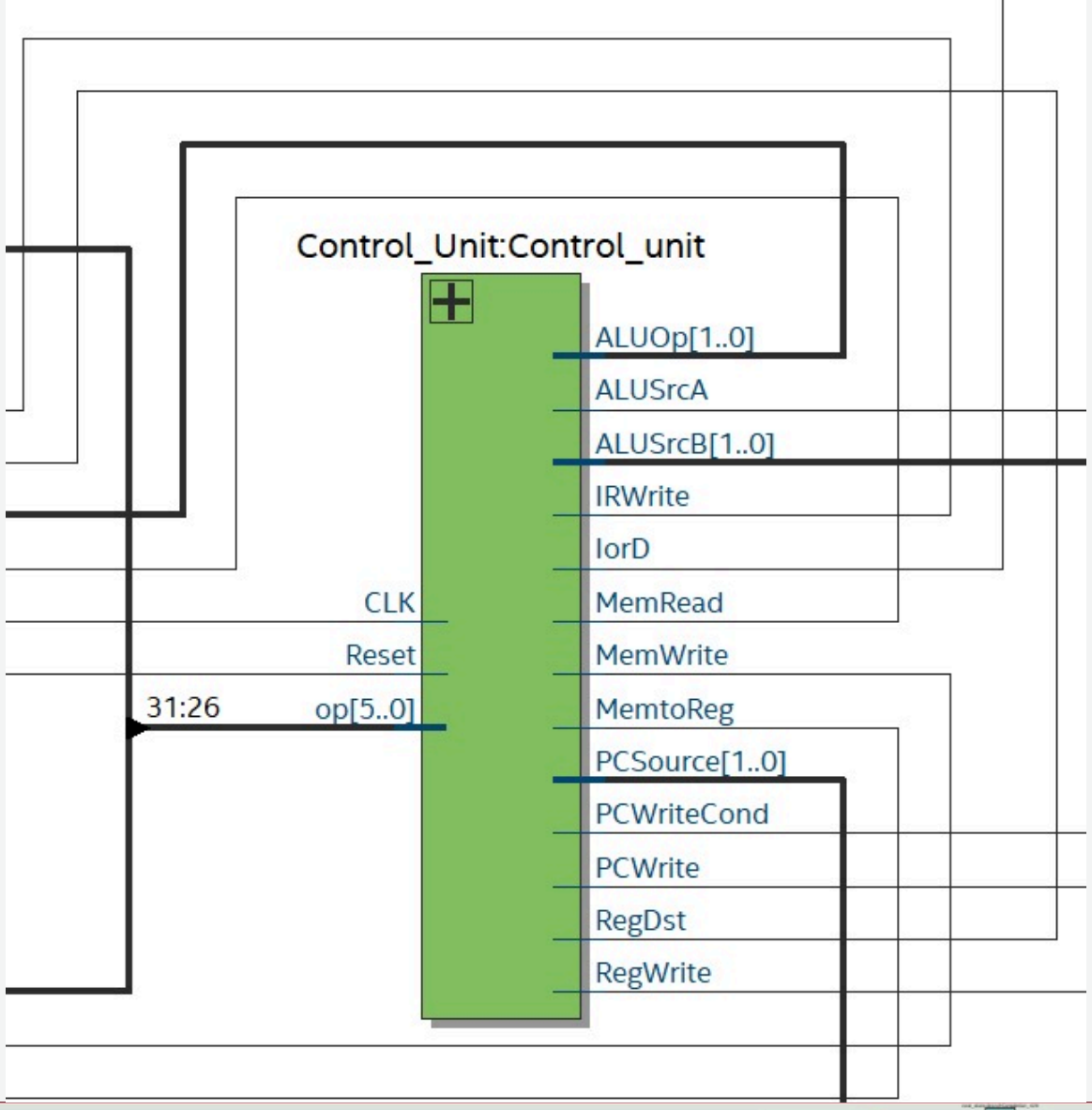
Block Diagram

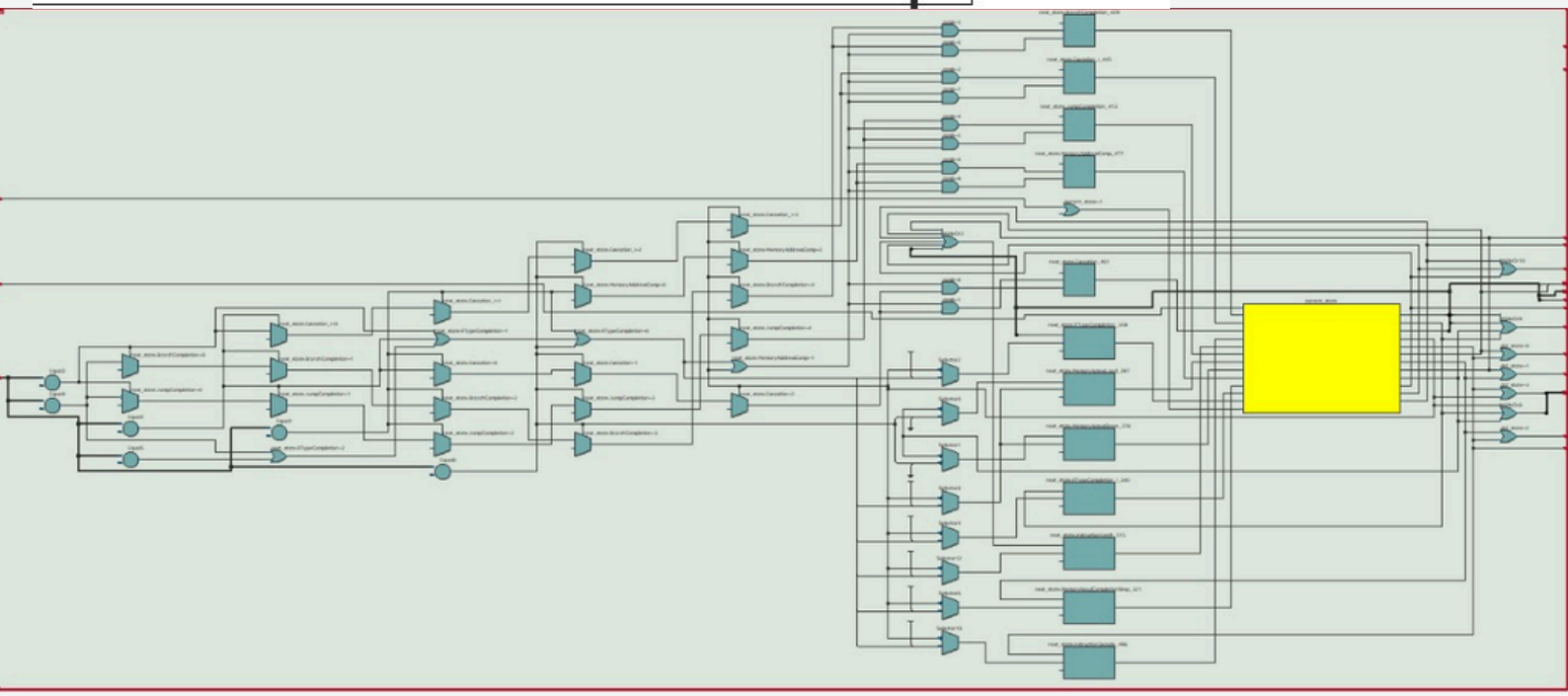


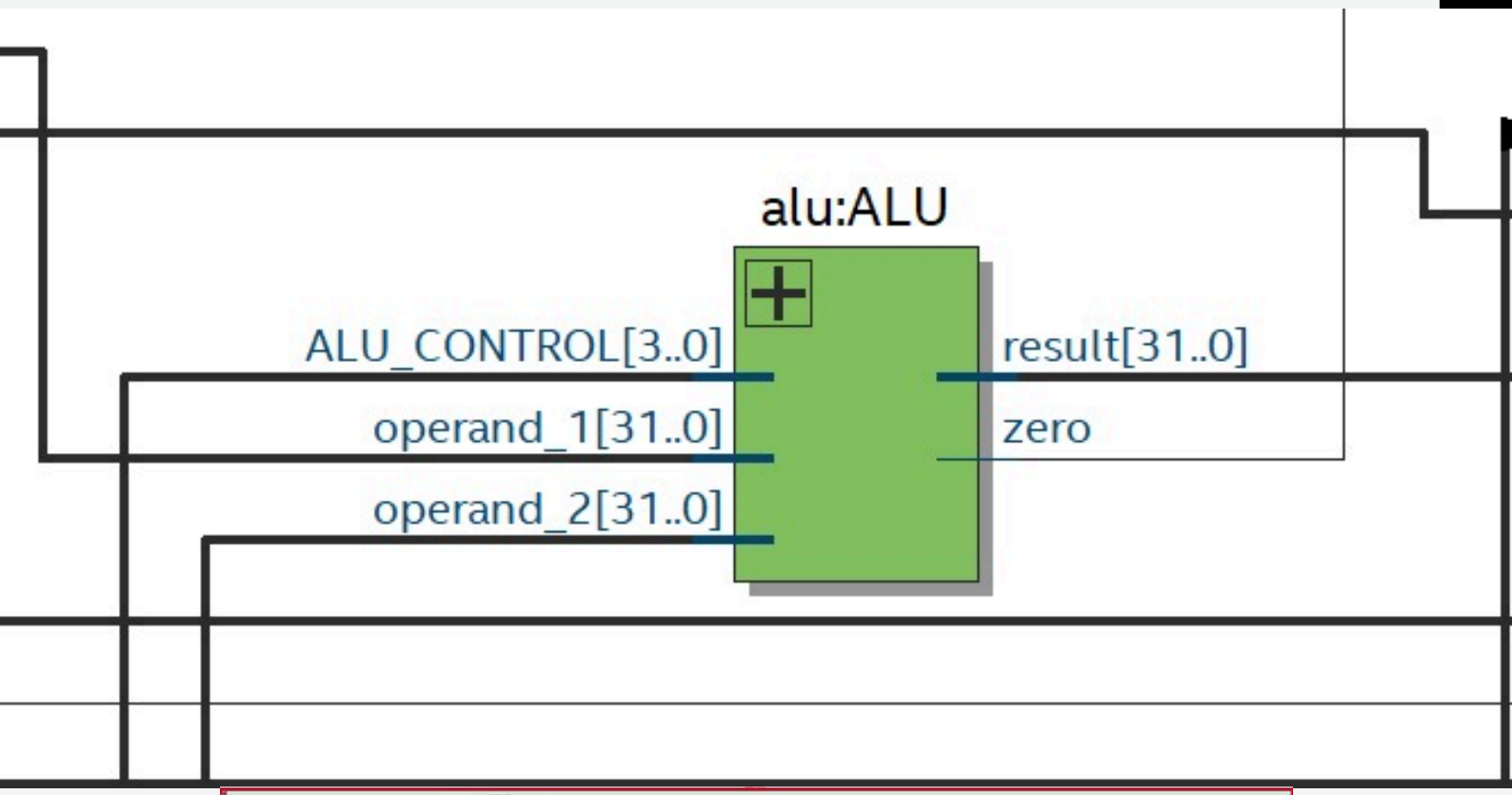
ALDEC	(C)ALDEC, Inc 2260 Corporate Circle Henderson, NV 89074
Created:	5/14/2025
Title:	No Title
Revision:	1.0
Page:	1 / 1

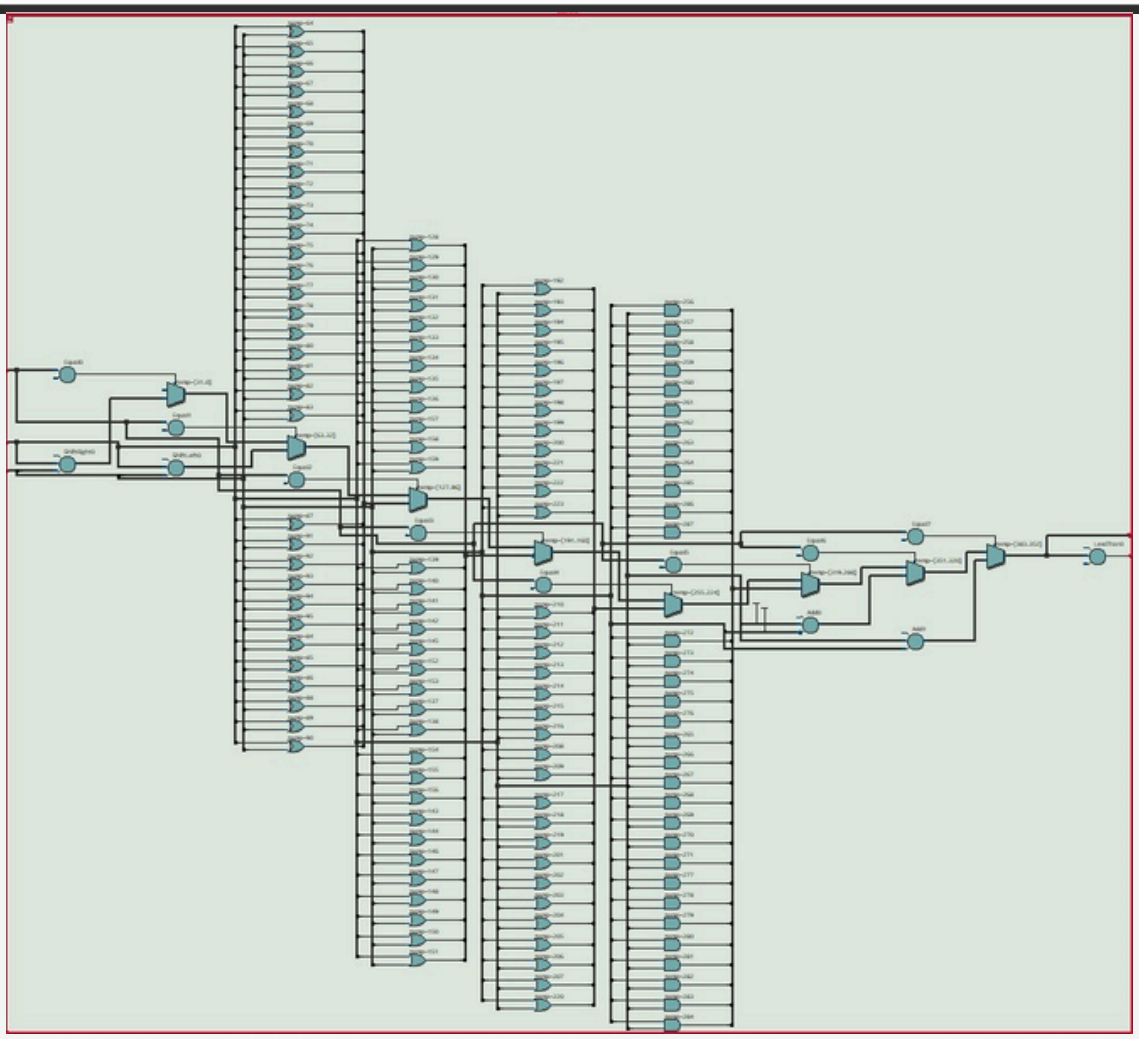
Schematic diagram

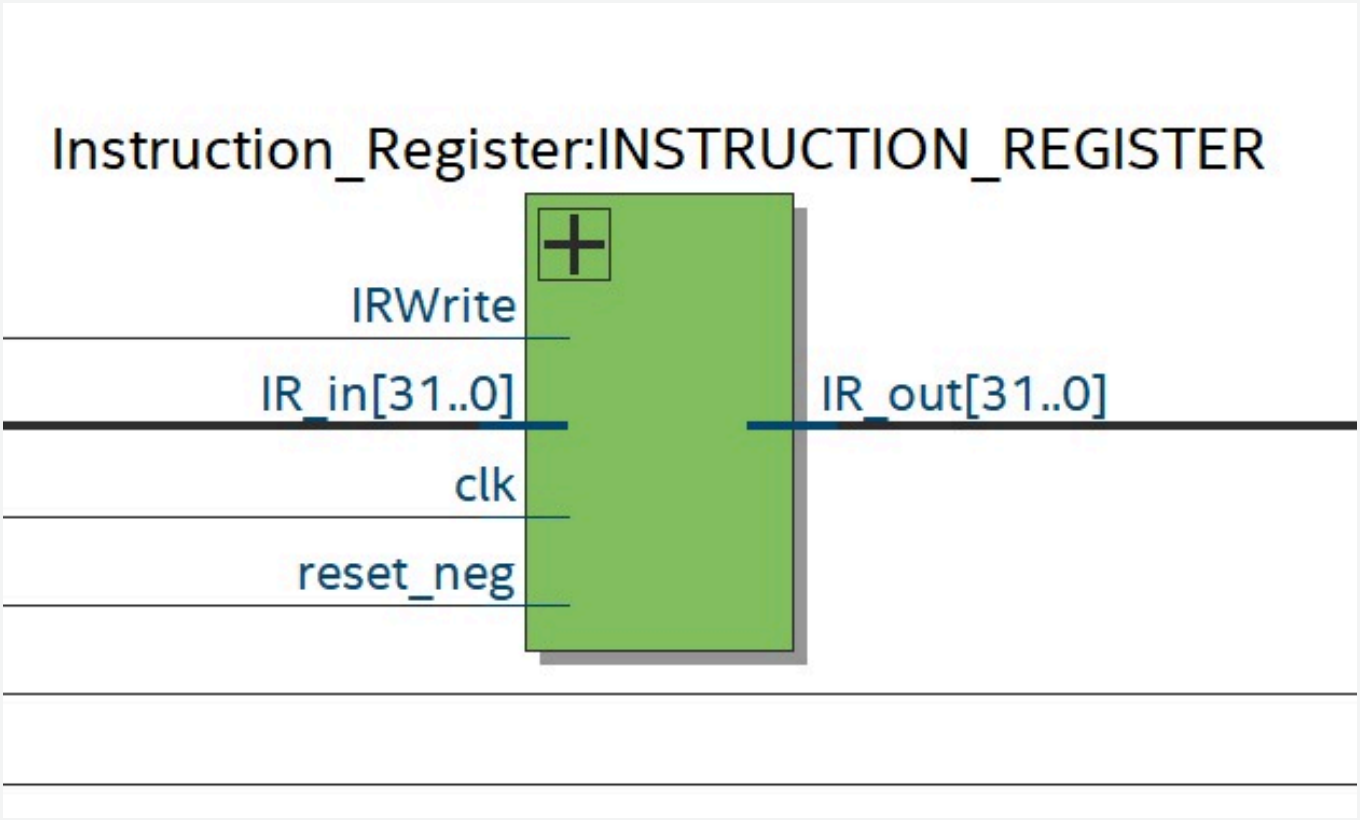


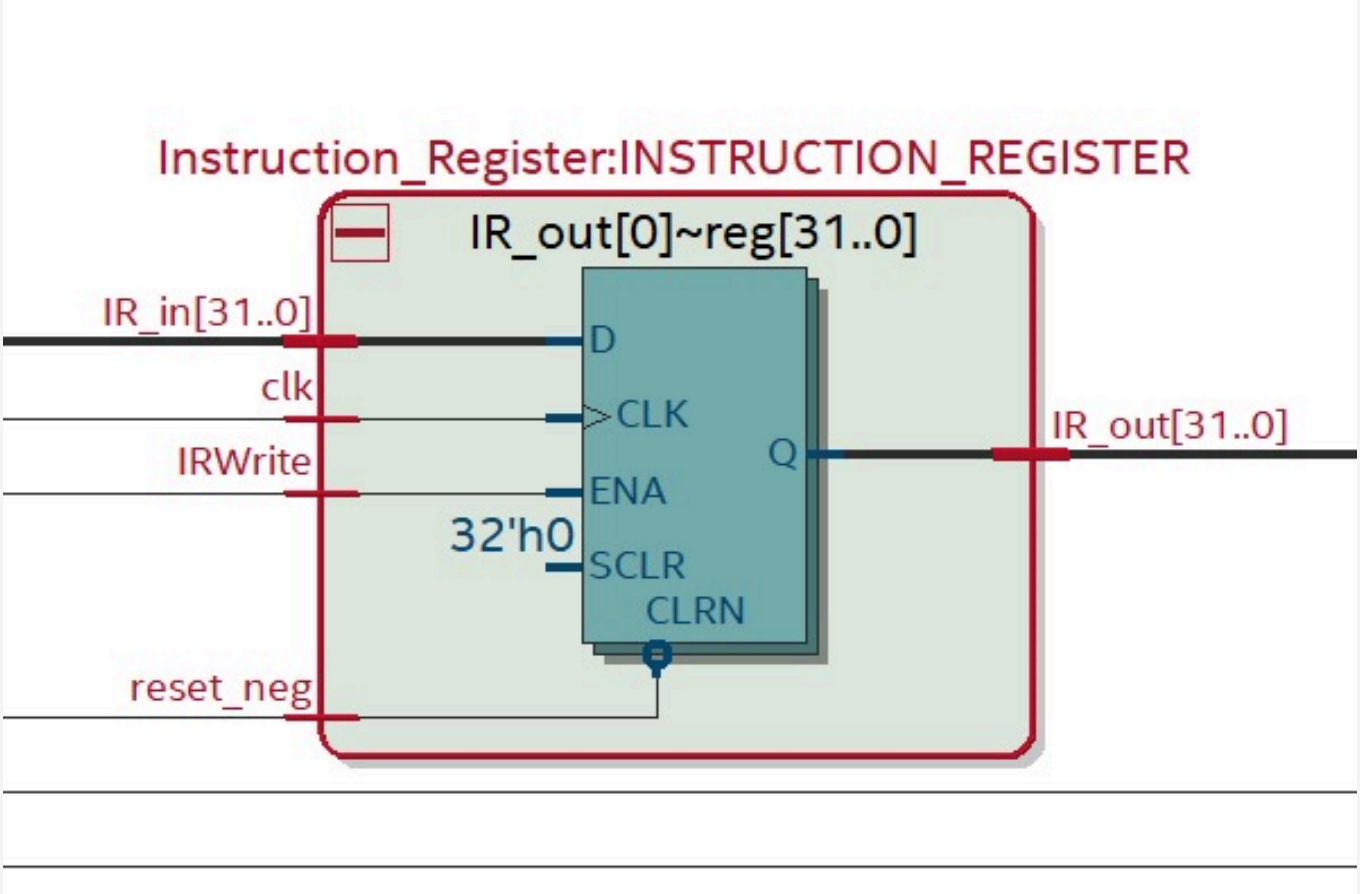

Control_Unit:Control_unit
ALUOp[1..0]
ALUSrcA
ALUSrcB[1..0]
IRWrite
IorD
CLK
MemRead
Reset
MemWrite
31:26
op[5..0]
MemtoReg
PCSource[1..0]
PCWriteCond
PCWrite
RegDst
RegWrite

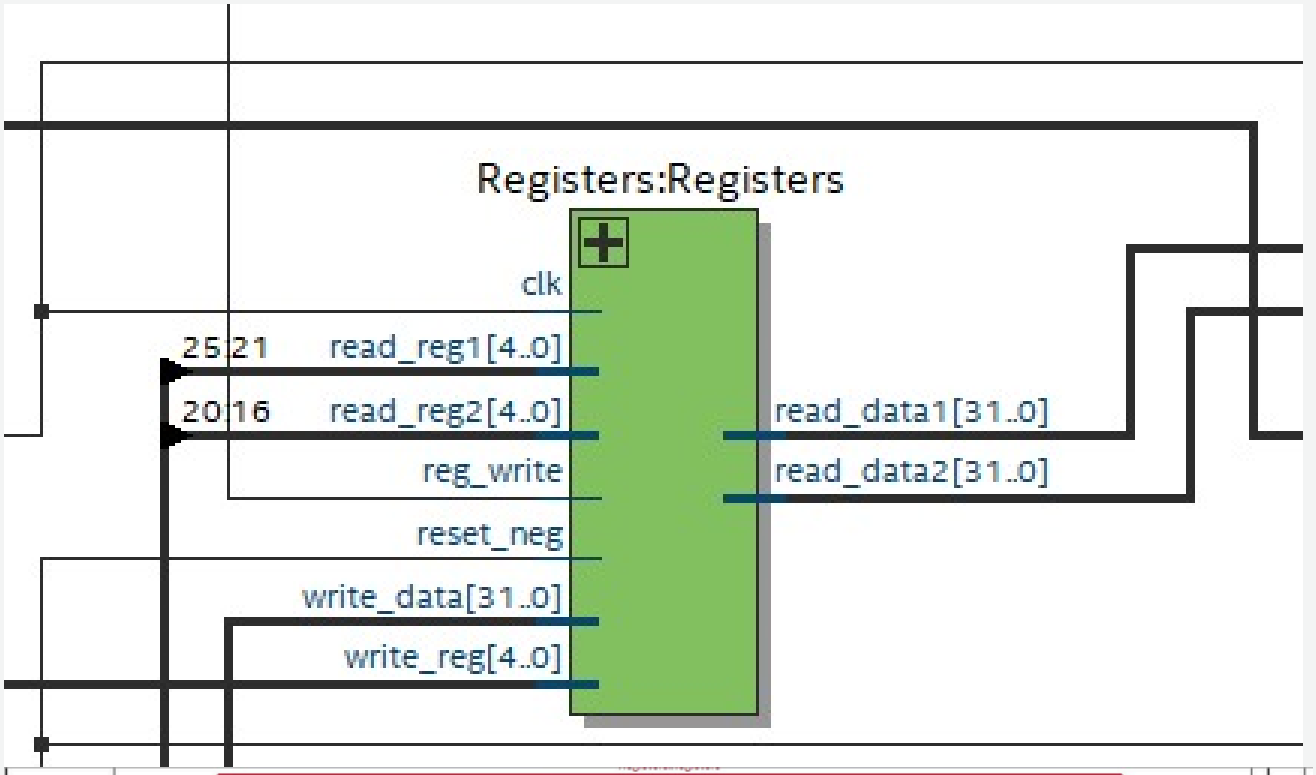


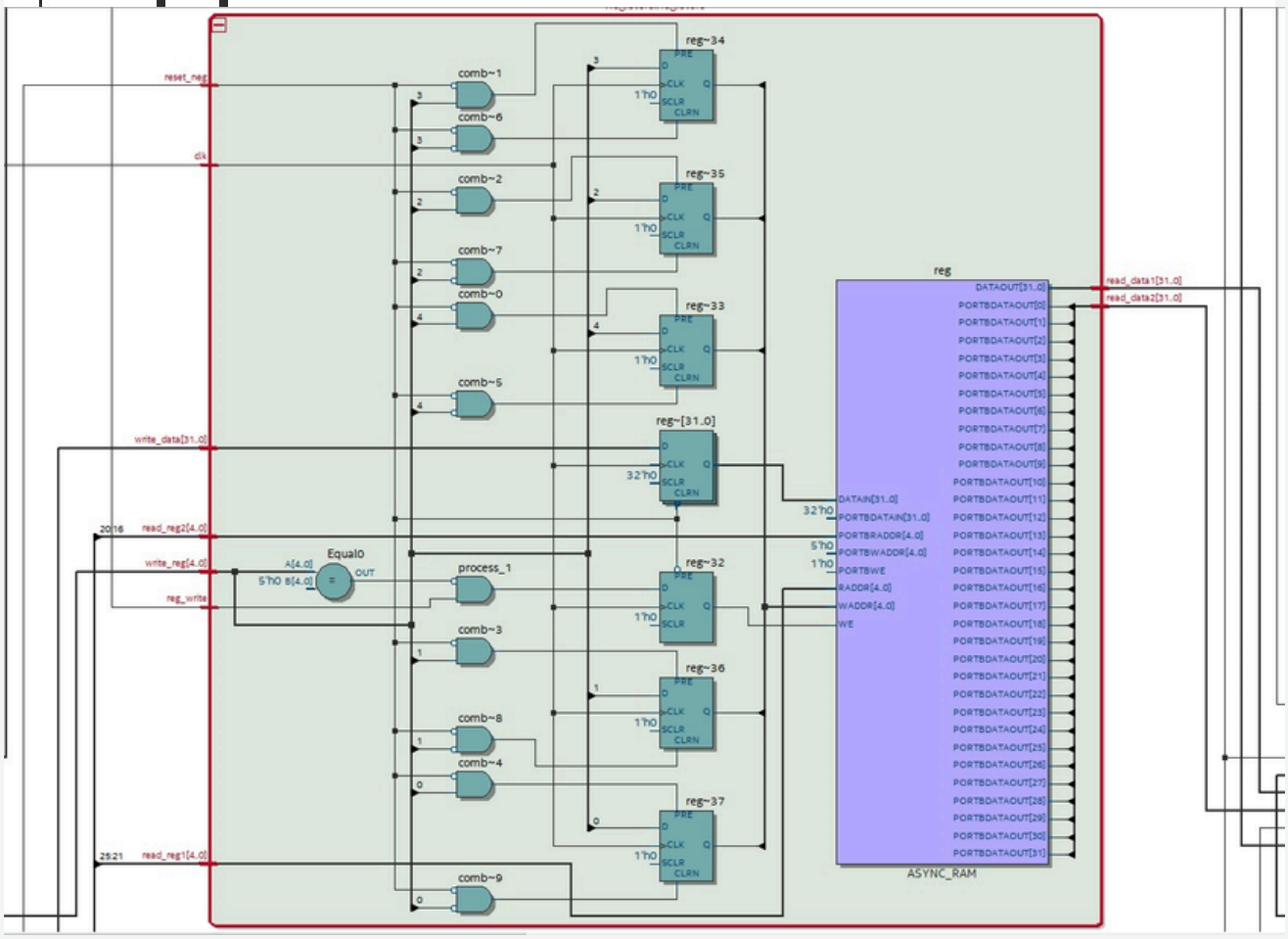

alu:ALU
ALU_CONTROL[3..0]
operand_1[31..0]
operand_2[31..0]
result[31..0]
zero

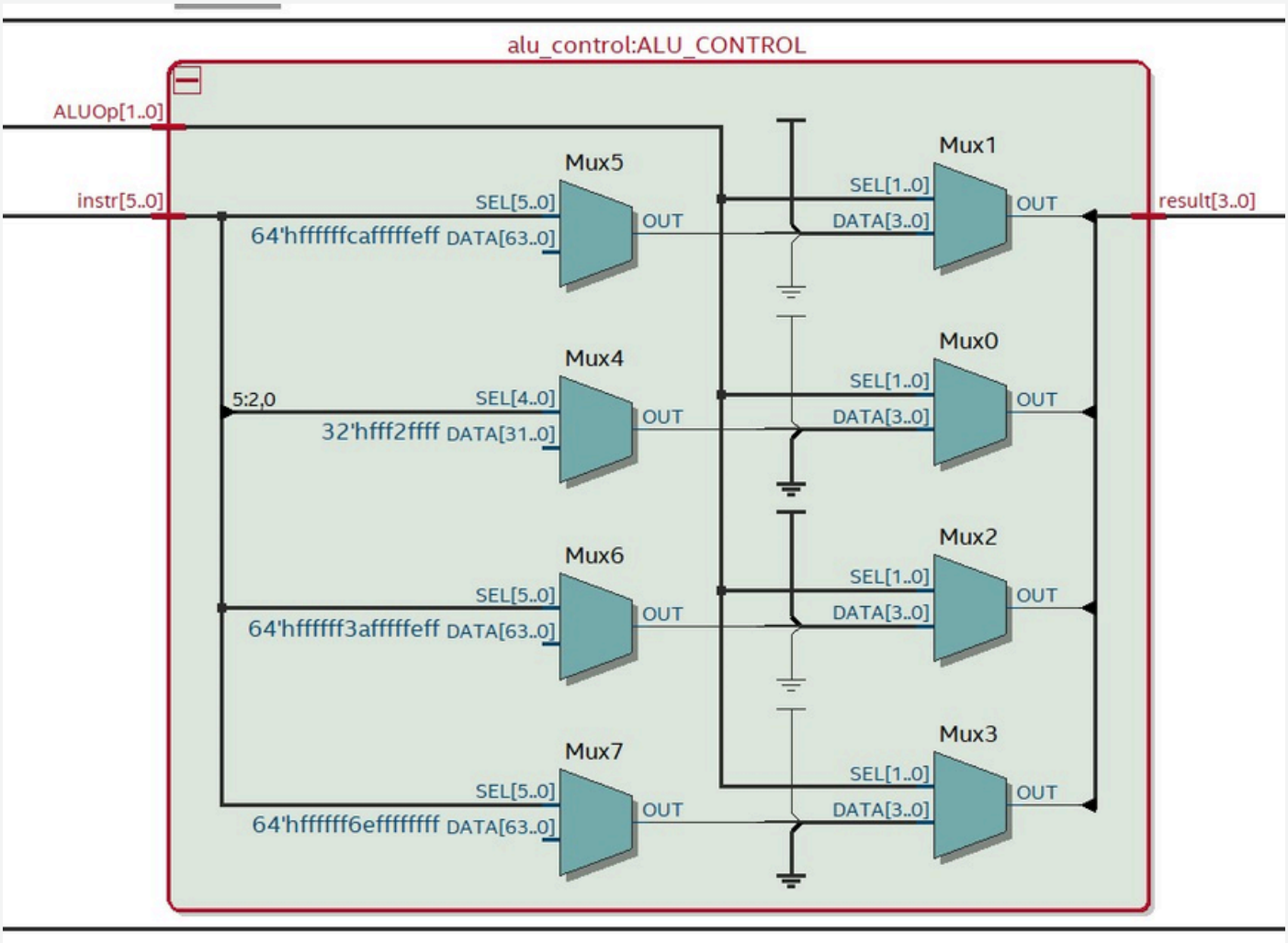


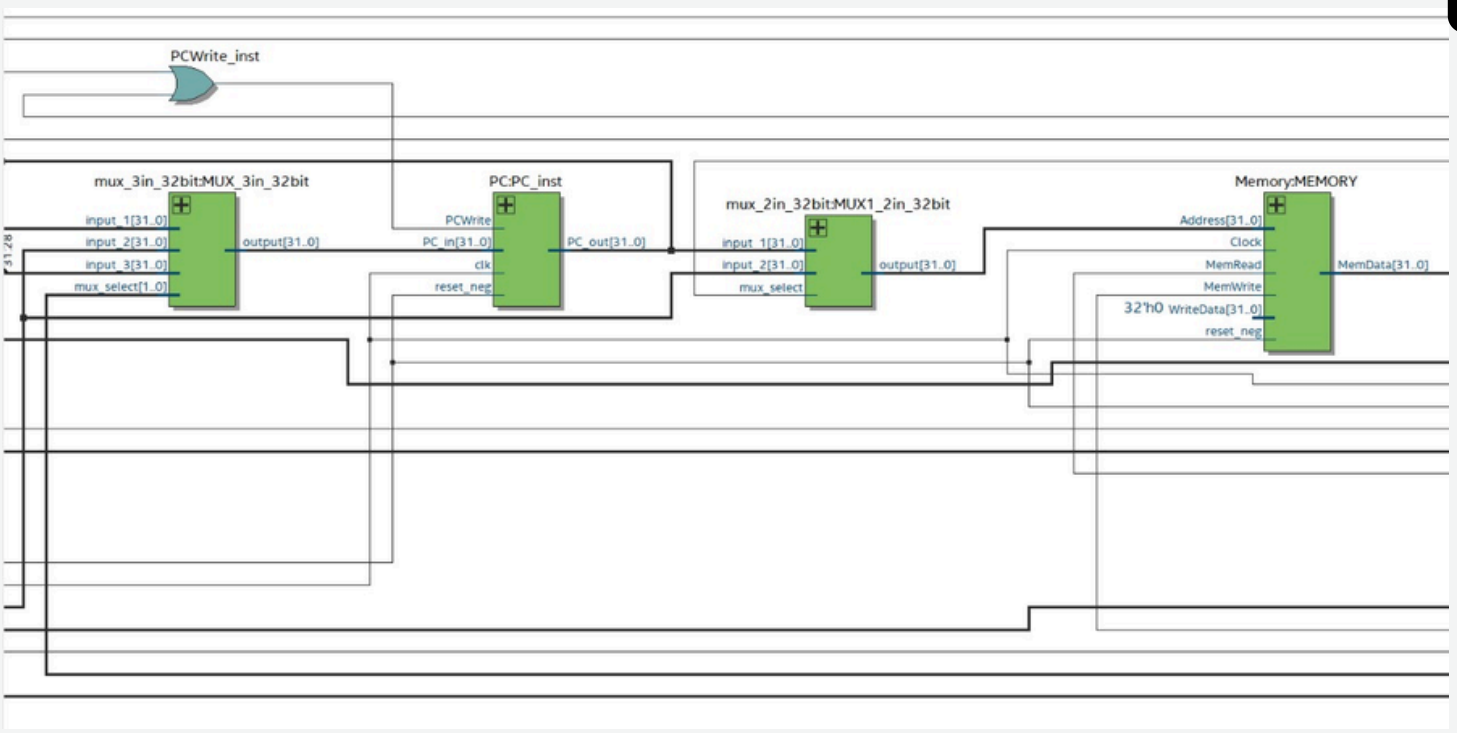

Instruction_Register:INSTRUCTION_REGISTER
IRWrite
IR_in[31..0]
clk
reset_neg
IR_out[31..0]

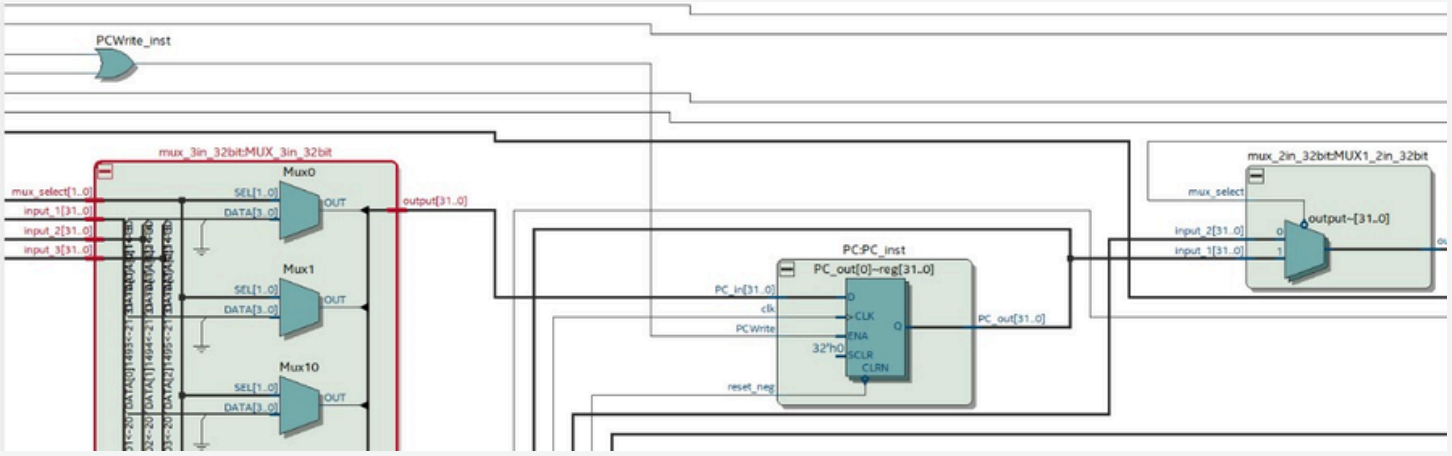

Instruction_Register:INSTRUCTION_REGISTER
IR_out[0]~reg[31..0]
IR_in[31..0]
clk
IRWrite
32'h0
reset_neg
D
CLK
ENA
SCLR
CLRN
Q
IR_out[31..0]


Registers:Registers
clk
25:21
read_reg1[4..0]
20:16
read_reg2[4..0]
reg_write
reset_neg
write_data[31..0]
write_reg[4..0]
read_data1[31..0]
read_data2[31..0]


reg~34
reg~35
reg~33
reg~[31..0]
reg~32
reg~36
reg~37
comb~1
comb~6
comb~2
comb~7
comb~0
comb~5
comb~3
comb~8
comb~4
comb~9
process_1
Equal0
reg
ASYNC_RAM
reset_neg
clk
write_data[31..0]
read_reg2[4..0]
write_reg[4..0]
reg_write
read_reg1[4..0]
read_data1[31..0]
read_data2[31..0]

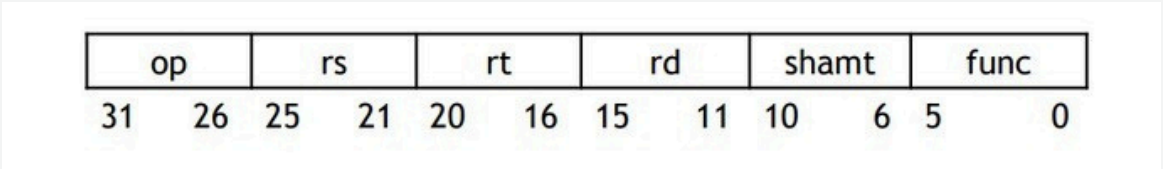

alu_control:ALU_CONTROL
ALUOp[1..0]
instr[5..0]
result[3..0]
Mux5
SEL[5..0]
64'hffffffcaffffeff DATA[63..0]
OUT
Mux1
SEL[1..0]
DATA[3..0]
OUT
Mux4
5:2,0
SEL[4..0]
32'hfff2ffff DATA[31..0]
OUT
Mux0
SEL[1..0]
DATA[3..0]
OUT
Mux6
SEL[5..0]
64'hffffff3affffeff DATA[63..0]
OUT
Mux2
SEL[1..0]
DATA[3..0]
OUT
Mux7
SEL[5..0]
64'hffffff6effffffff DATA[63..0]
OUT
Mux3
SEL[1..0]
DATA[3..0]
OUT


PCWrite_inst
mux_3in_32bit:MUX_3in_32bit
input_1[31..0]
input_2[31..0]
input_3[31..0]
mux_select[1..0]
output[31..0]
PC:PC_inst
PCWrite
PC_in[31..0]
clk
reset_neg
PC_out[31..0]
mux_2in_32bit:MUX1_2in_32bit
input_1[31..0]
input_2[31..0]
mux_select
output[31..0]
Memory:MEMORY
Address[31..0]
Clock
MemRead
MemWrite
32'h0 WriteData[31..0]
reset_neg
MemData[31..0]


PCWrite_inst
mux_3in_32bit:MUX_3in_32bit
Mux0
Mux1
Mux10
SEL[1..0]
DATA[3..0]
OUT
mux_select[1..0]
input_1[31..0]
input_2[31..0]
input_3[31..0]
output[31..0]
PC:PC_inst
PC_out[0]~reg[31..0]
PC_in[31..0]
clk
PCWrite
32'h0
reset_neg
PC_out[31..0]
mux_2in_32bit:MUX1_2in_32bit
mux_select
input_2[31..0]
input_1[31..0]
output~[31..0]

Instruction type

R-type



add $4 ,$2,$1

(0) Decimal	$1	$2	$4	shamt	Function (add)
000000	00001	00010	00100	00000	100 000

x “ 00412020 ”

sub $6 ,$1,$5

(0) Decimal	$1	$5	$6	shamt	Function (Sub)
000000	00001	00101	00110	00000	100 010

x “00053022”

and $4 ,$7,$13

(0) Decimal	**$4**	$7	$13	shamt	Function (and)
000 000	00111	01101	00100	00000	100 100

x “ 01B42024”

nand $4 ,$7,$13

op (0) Decimal	**$4**	$7	$13	shamt	Function (nand)
000 000	00111	01101	00100	00000	001 111

x “01B42037”

or $1 ,$4,$5

op (0) Decimal	**$1**	$4	$5	shamt	Function (or)
000 000	00100	00101	00001	00000	100 101

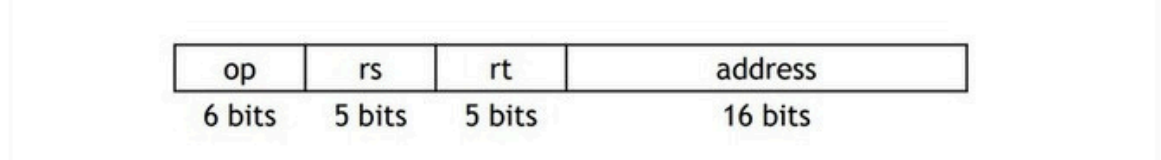
x “00452025”

set on less than $4 ,$2,$3

(0) Decimal	**$2**	$3	$4	shamt	Function (slt)
000 000	00010	00011	00100	00000	101 010

x “0022182A”

I-type



LW $2,2($0)

35 Decimal	$0	$2	“address” 2
100011	00000	00010	0000000000000010

x “ 8c020002 ”

SW $4,0($5)

43 Decimal	$5	$4	“address” 0
101011	00101	00100	0000000000000000

x “AD240000”

add i $5,$5,1

8 Decimal	$5	$2	“immediant” 1
001000	00101	00101	0000000000000001

x “21310001”

beq $10,$s11,8

4 Decimal	$10	$11	“address” 1
000100	01010	01011	0000000000001000

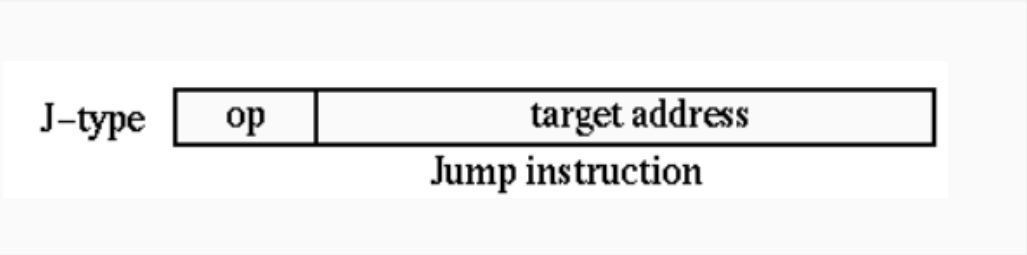
x “014B1008”

bneq $3,$0,-8

5 Decimal	$3	$0	“address” 8
000101	00011	00000	1111111111111000

x “1130FFF8”

J-type



j 16

35 Decimal	Target address
000010	00000000000000000000010000

x “08000010”

ALU Control

Op Code	Inputs Function (6bit)	inputs Op Code (2 bit)	Operation	Outputs Alu_Ctrl
Load word	xxxxxx	00	add	0000
Store word	xxxxxx	00	add	0000
bneq	xxxxxx	01	subt	0110
beq	xxxxxx	01	subt	0110
R Format	xxxxxx	11	Invalid / default	1111
	100 000	10	ADD	0000
	100 010	10	SUB	0001
	100 100	10	AND	0010
	100 101	10	OR	0011
	100 111	10	NOR	0100
	111 000	10	XOR	0101
	001000	10	ADDI	1001
	xxxxxx	10	SLL	0111
	xxxxxx	10	SRL	1000

Control Value

State	PCWrite	PCWriteCond	IorD	MemRead	MemWrite	MemToReg	IRWrite	PCSource	ALUOp	ALUSrcB	ALUSrcA	RegWrite	RegDst
Instruction Fetch	1	0	0	1	0	0	1	00	00	01	0	0	0
Instruction Decode	0	0	0	0	0	0	0	00	00	11	0	0	0
Memory Address Comp.	0	0	0	0	0	0	0	00	00	10	1	0	0
Execution (R-type)	0	0	0	0	0	0	0	00	10	00	1	0	0
Execution I (addi)	0	0	0	0	0	0	0	00	00	10	1	0	0
Branch Completion	0	1	0	0	0	0	0	01	01	00	1	0	1
Jump Completion	1	0	1	0	0	0	0	10	00	00	0	0	0
Memory Access (Load)	0	0	1	1	0	0	0	00	00	00	0	0	0
Memory Access (Store)	0	0	1	0	1	0	0	00	00	00	0	0	0
R-Type Completion	0	0	0	0	0	0	0	00	00	00	0	1	1
R-Type Completion I	0	0	0	0	0	0	0	00	00	00	0	1	0
Mem Read Completion Step	0	0	0	1	0	1	0	00	00	00	0	1	0

ALU

Inputs			Outputs	
input 1	input 2	Alu_Ctrl	Result	Zero
x	x	0010	input1 + input2	0
x	x	0001	input1 - input2	0
x	x	0010	input1 AND input2	0
x	x	0011	input1 OR input2	0
x	x	0011	input1 XOR input2	0
x	x	0111	input1 << input2	0
x	x	1000	input1 >> input2	0
x	x	others	(others => '0')	1

ALU

Inputs			Outputs	
input 1	input 2	Alu_Ctrl	Result	Zero
x	x	0010	input1 + input2	0
x	x	0001	input1 - input2	0
x	x	0010	input1 AND input2	0
x	x	0011	input1 OR input2	0
x	x	0011	input1 XOR input2	0
x	x	0111	input1 << input2	0
x	x	1000	input1 >> input2	0
x	x	others	(others => '0')	1

References

- Book:
  - Processor Implementation in VHDL
- GitHub:
  - https://github.com/david-palma/mips-32bit/tree/master/MIPS32_multi_cycle/code
- YouTube:
  - **R Type, I Type, J Type - The Three MIPS Instruction Formats**
    - https://www.youtube.com/watch?v=luH_8t7vPu8&t=3s&ab_channel=TahiaTabassum
  - **Multi-Cycle Data Path**
    - https://www.youtube.com/watch?v=hmK3xabU6Ts&ab_channel=NachumDanzig
  - **SIngle-cycle Processor explanation with code**
    - https://www.youtube.com/watch?v=_z20VuxRzsc&list=PL6EKgIJmvYfzk2AsZvqxkqDeNMV5nXAJI&index=3&ab_channel=twalsh123
  - **The MIPS Processor (Computer Organization)**
    - https://www.youtube.com/watch?v=vtE-6bIULYg&ab_channel=AhmedHekal